Brian Urbina, Shantonu Mandal, Alex McGee
WES 237C

Project 3 Report: Discrete Fourier Transform

Question 1 - DFT32 Baseline

Part (a)

Yes, changing the accuracy (bit width) of a custom CORDIC core would impact the resource usage. Increasing the number of bits would use more resources. A DFT32 needs 32 x 32 = 1024 sin/cos calculations. So any additional CORDIC latency will be multiplied across a single transform - thus lowering the DFT throughput. Also, any error in the CORDIC will carry into the DFT output.

Brian Urbina, Shantonu Mandal, Alex McGee
WES 237C

Question 2 - DFT32 Table Lookup

Part (a)

	Baseline (Q1)	**Table Lookup**	**Change**
Latency (clock cycles)	54884 - 55908	257	⬇️ ~214x
Initiation Interval (II) (clock cycles)	54885 - 55909	258	⬇️ ~213x
Throughput (DFT/s)	1790 - 1820	387,000	⬆️ ~214x
BRAM	2	66	⬆️ 33x
DSP	53	640	⬆️ ~12x
FF	5275	59208	⬆️ ~11x
LUT	9111	91730	⬆️ ~10x

Brian Urbina, Shantonu Mandal, Alex McGee
WES 237C

Question 3 - DFT32 Interface Change

Part (a)

Accepting the output array as an input to dft() (it will translate to a separate input port/stream in DFT core) avoids doing an in-place DFT. So, the HLS function no longer reads and writes to the same location. And helps do away with the temporary buffer and copying back the output loop.

With separate arrays, we can partition each one independently - getting a lower II. It also reduces latency since we can get rid of the output copy loop at the end of the baseline version.

Part (b)

	Table Lookup (in-place) (Q2)	**Interface Change**	**Change %**
Latency (clock cycles)	257	221	⬇️ 14%
Initiation Interval (II) (clock cycles)	258	222	⬇️ ~14%
Throughput (DFT/s)	387,000	450,500	⬆️ 16%
BRAM	66	64	⬇️ 3%
DSP	640	640	0
FF	59208	59189	⬇️ ~0.03%
LUT	91730	91521	⬇️ ~0.2%

Part (c)

With the interface change, latency and II dropped by 14% (and throughput increased, correspondingly). This is likely due to the output copy loop removal.
Resource usage stayed roughly the same.

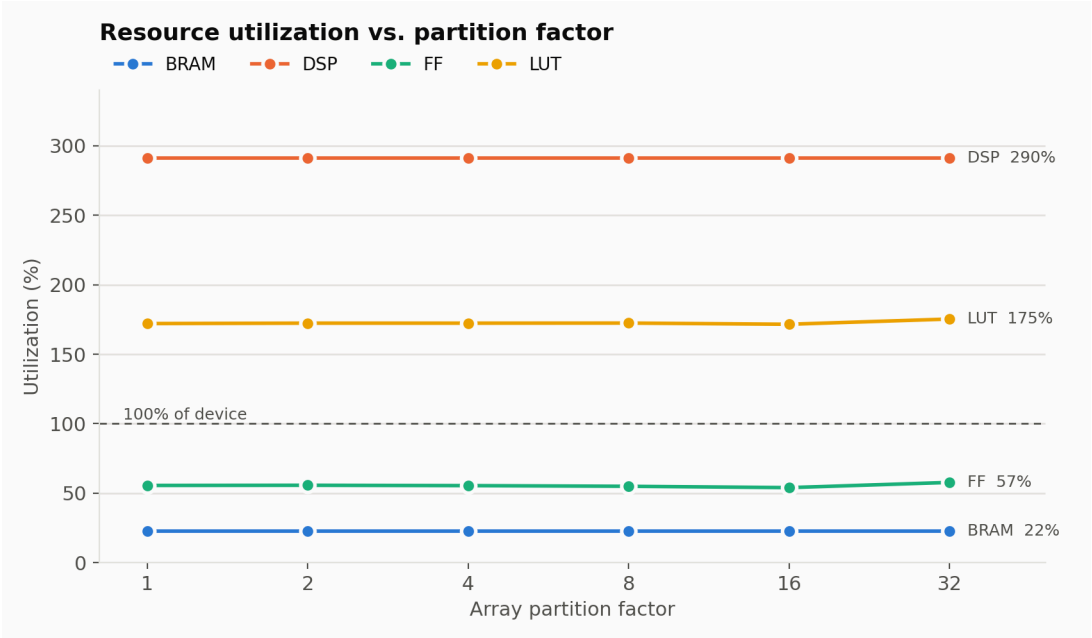
Brian Urbina, Shantonu Mandal, Alex McGee
WES 237C

Question 4 - DFT32 Array Partitioning

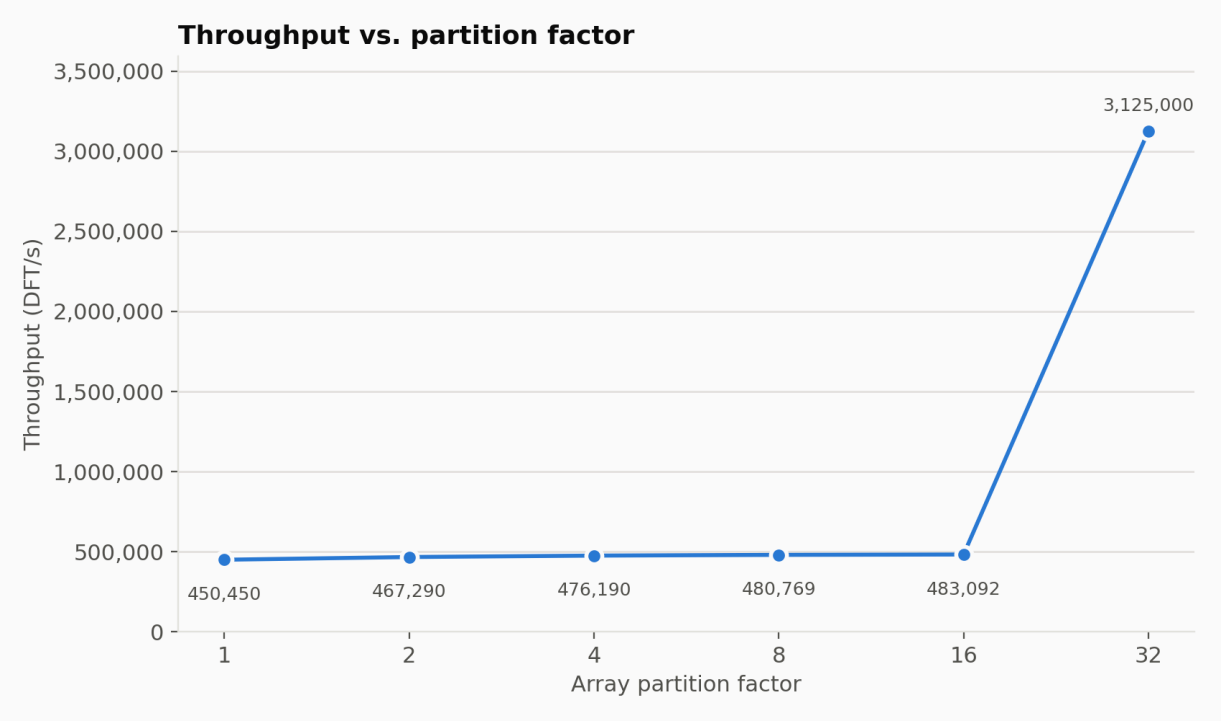
Part (a)

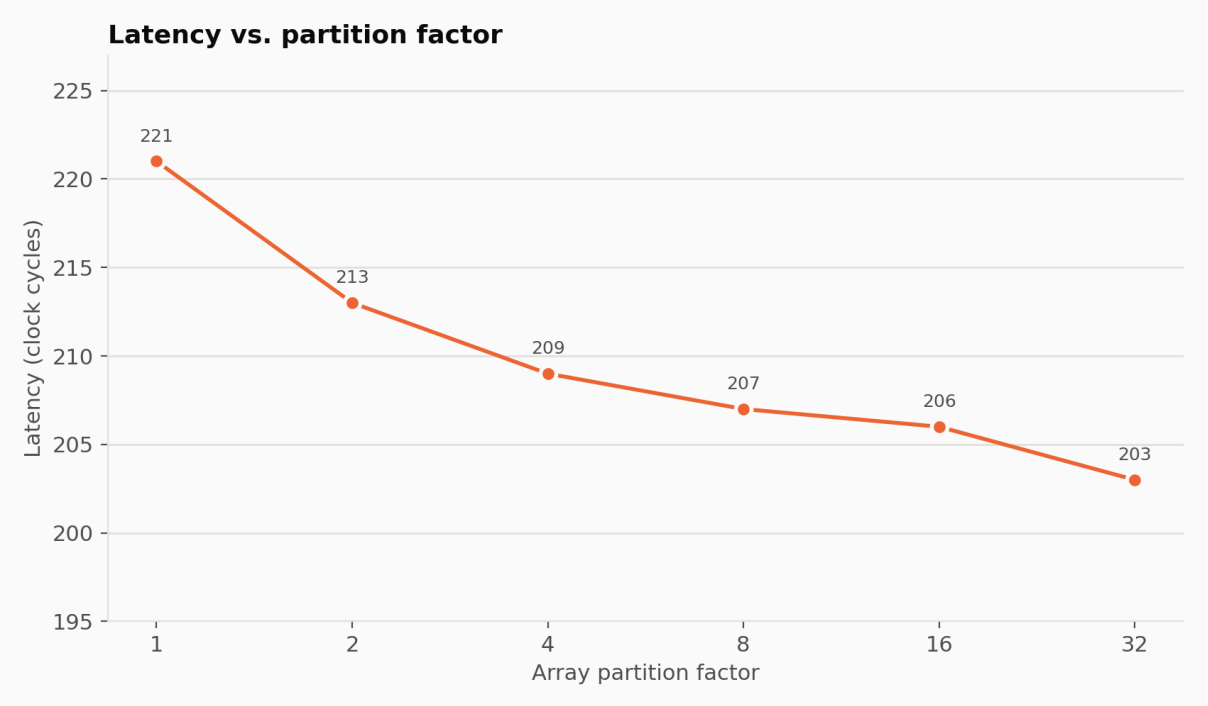
	Factor = 1	Factor = 2	Factor = 4	Factor = 8	Factor = 16	Factor = 32
Latency (clock cycles)	221	213	209	207	206	203
Initiation Interval (II) (clock cycles)	222	214	210	208	207	32
Throughput (DFT/s)	450,500	467,290	476,190	480,769	483,092	3,125,000
BRAM	64	64	64	64	64	64
DSP	640	640	640	640	640	640
FF	59189	59314	59054	58540	57515	61509
LUT	91521	91675	91669	91705	91252	93250

Part (b)



Part (c)


Throughput vs. partition factor
Throughput (DFT/s)
3,500,000
3,000,000
2,500,000
2,000,000
1,500,000
1,000,000
500,000
0
450,450
467,290
476,190
480,769
483,092
3,125,000
1
2
4
8
16
32
Array partition factor


Latency vs. partition factor
Latency (clock cycles)
225
220
215
210
205
200
195
221
213
209
207
206
203
1
2
4
8
16
32
Array partition factor

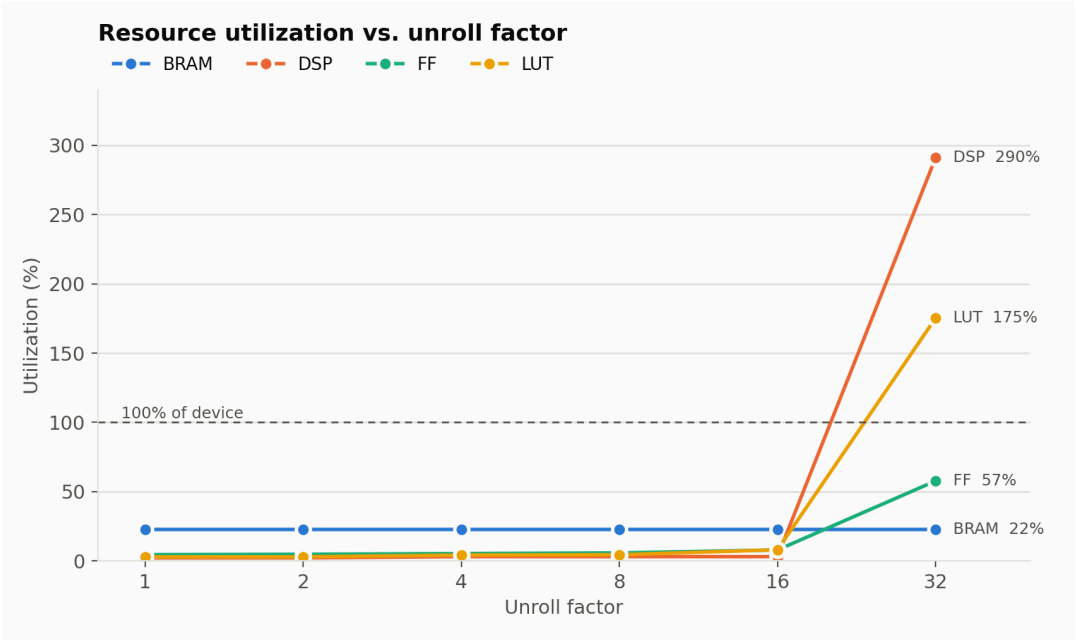
Brian Urbina, Shantonu Mandal, Alex McGee
WES 237C

Question 5 - Loop Unrolling

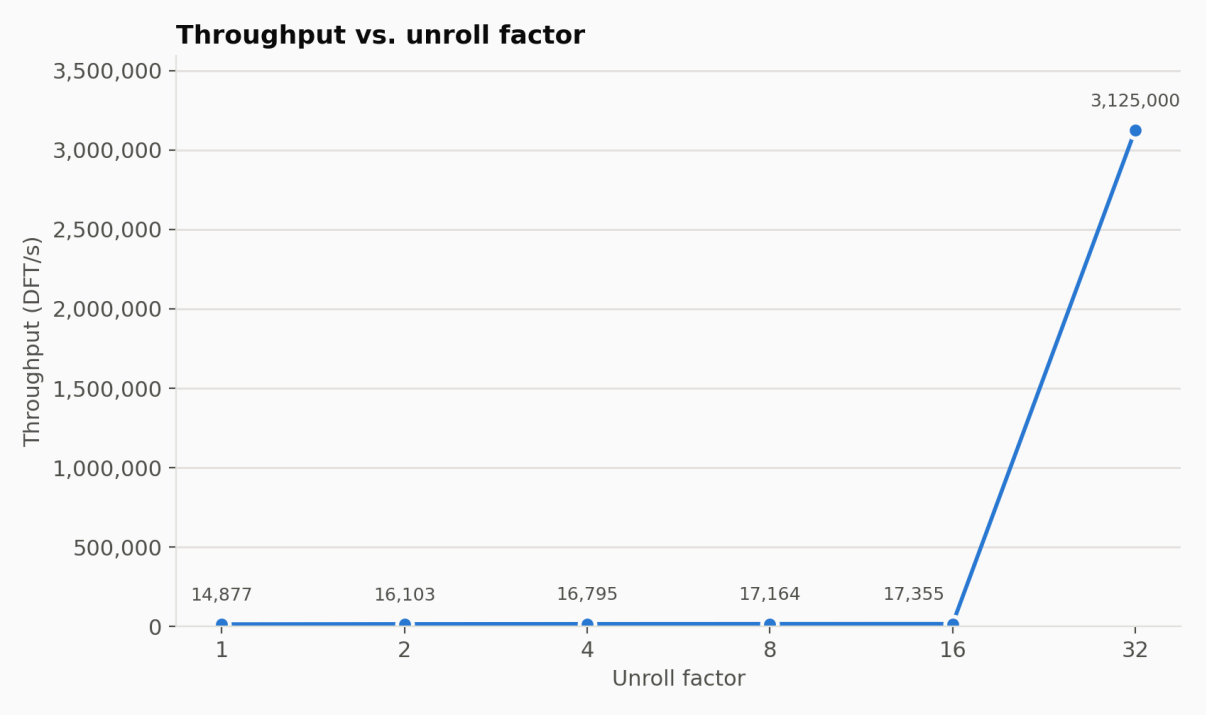
Part (a)

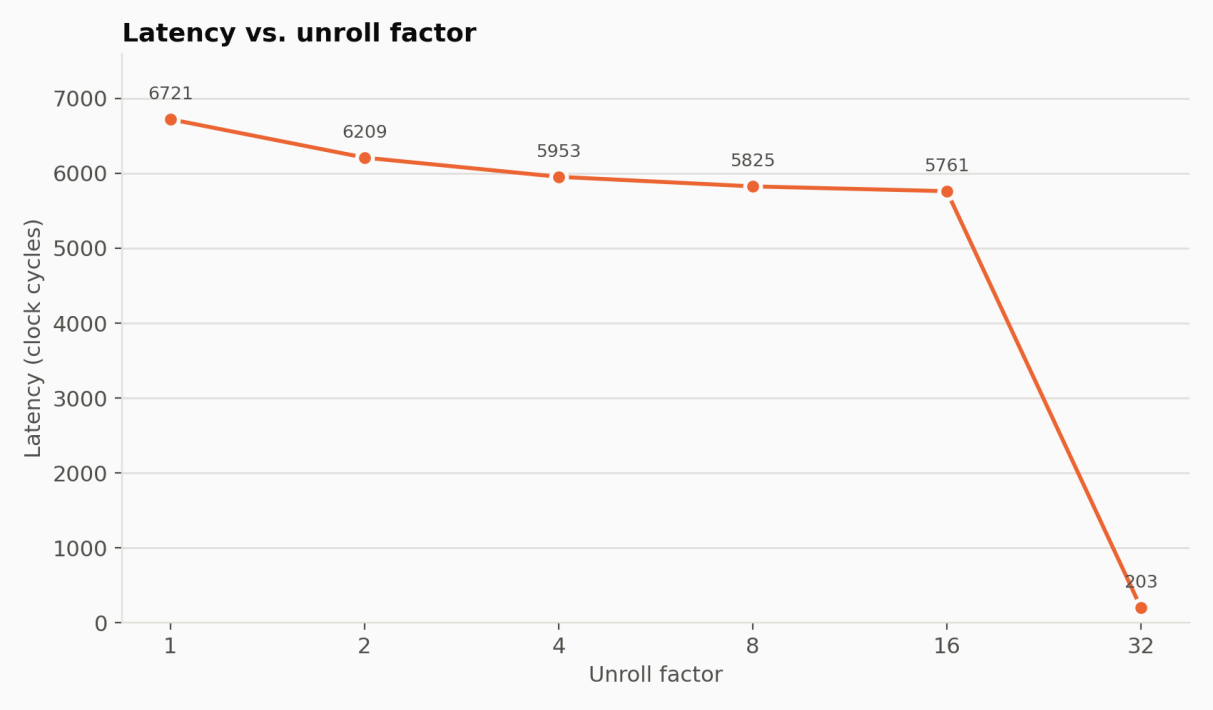
	Factor = 1	Factor = 2	Factor = 4	Factor = 8	Factor = 16	Factor = 32
Latency (clock cycles)	6721	6209	5953	5825	5761	203
Initiation Interval (II) (clock cycles)	6722	6210	5954	5826	5762	32
Throughput (DFT/s)	14,877	16,103	16,795	17,164	17,355	3,125,000
BRAM	64	64	64	64	64	64
DSP	5	5	7	7	7	640
FF	4904	5195	5727	6259	8507	61509
LUT	1596	1621	2295	2413	4339	93250

Part (b)



Part (c)


Throughput vs. unroll factor
Throughput (DFT/s)
3,500,000
3,000,000
2,500,000
2,000,000
1,500,000
1,000,000
500,000
0
14,877
16,103
16,795
17,164
17,355
3,125,000
1
2
4
8
16
32
Unroll factor


Latency vs. unroll factor
Latency (clock cycles)
7000
6000
5000
4000
3000
2000
1000
0
6721
6209
5953
5825
5761
203
1
2
4
8
16
32
Unroll factor

Question 6 - DFT1024 Baseline

Part (a)

	Cyclic factor = 2	**Cyclic factor = 4**	**Cyclic factor = 2 (only shift_reg)**	**Cyclic factor = 2 (only c)**	**Complete**	**Block**
Latency (clock cycles)	140	208	140	282	139	206
Initiation Interval (II) (clock cycles)	141	209	141	283	140	207
Throughput (kHz)	1,025.5	679.6	1,025.5	506.4	1,019.5	670.4
BRAM	4	8	5	6	0	4
DSP	4	4	4	36	4	4
LUT	502	1043	492	8462	6591	
FF	624	964	668	11273	4806	769

Part (b)

	Disabled loop unrolling	**Disabled array partitioning**
Latency (clock cycles)	267	282
Initiation Interval (II) (clock cycles)	268	283
Throughput (kHz)	517.8	506.4
BRAM	4	7
DSP	2	36
LUT	468	8452
FF	384	11314

Question 7 - Best Design

Part (a)

In order to meet the necessary throughput, for our optimizations we used function-level initiation interval pipelining in order to get a new input every cycle. As a result, loop unrolling was automatically applied. We used cyclic array partitioning so every sample can be read and shifted in the same cycle. We split the shifting (TDL) and multiply and accumulate (MAC) using loop partitioning so each operation can be optimized independently. We also used bitwidth reduction using a bitwidth of 16-bits to use the smallest data type for our FIR which makes every adder and register smaller.

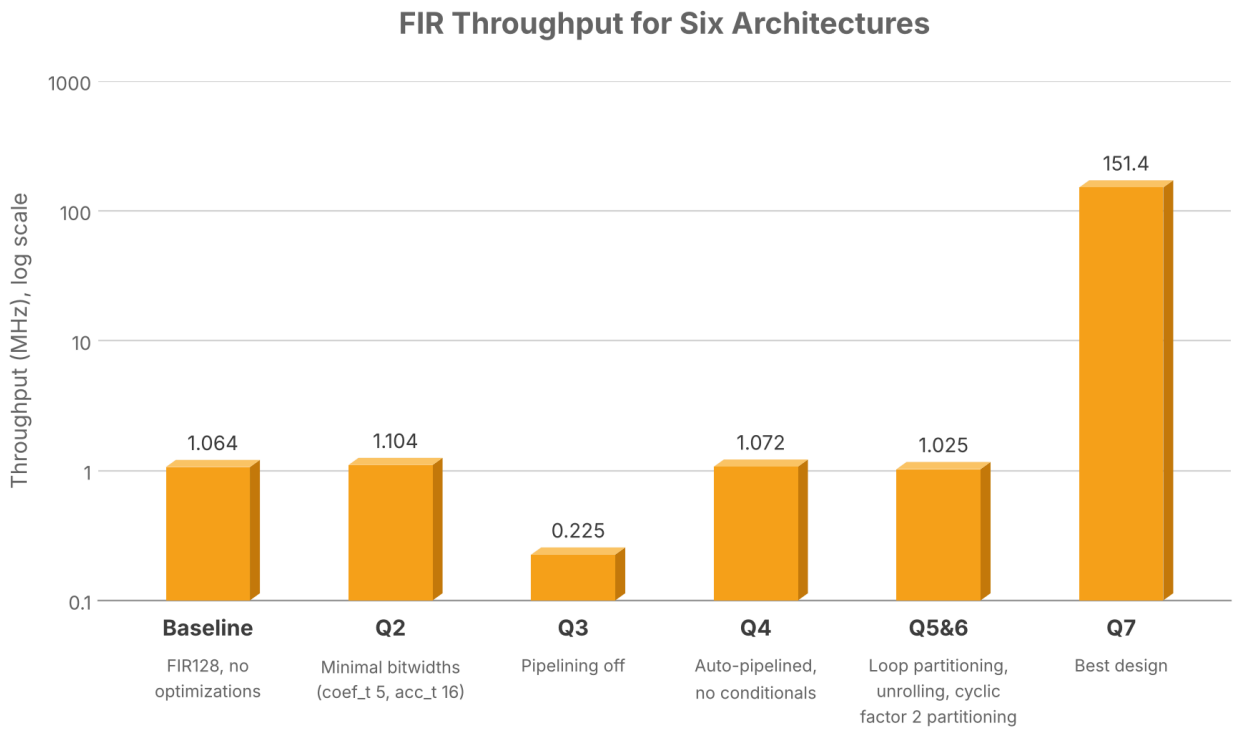
Latency (clock cycles)	**Initiation Interval (II) (clock cycles)**	**Throughput (MHz)**
29	1	151.446

Part (b)

BRAM	**DSP**	**FF**	**LUT**
0	1	9,553	4,604

Our best design computes all 128 taps in parallel and keeps the samples in 128 individual registers so that every one can be read and shifted in the same cycle. Pipelining the function to accept a new sample every cycle also adds registers between its 29 stages, which requires additional flip-flops. In exchange for a throughput of 151.4 MHz compared with 1.104 MHz,
The design uses 9,553 flip-flops and 4,604 LUTs compared with 244 and 239 for the baseline.

Throughput Chart



Question 8 - Github Repo

https://github.com/UCSD-237C-2026/wes-237c-project-1-fir-group-2.git